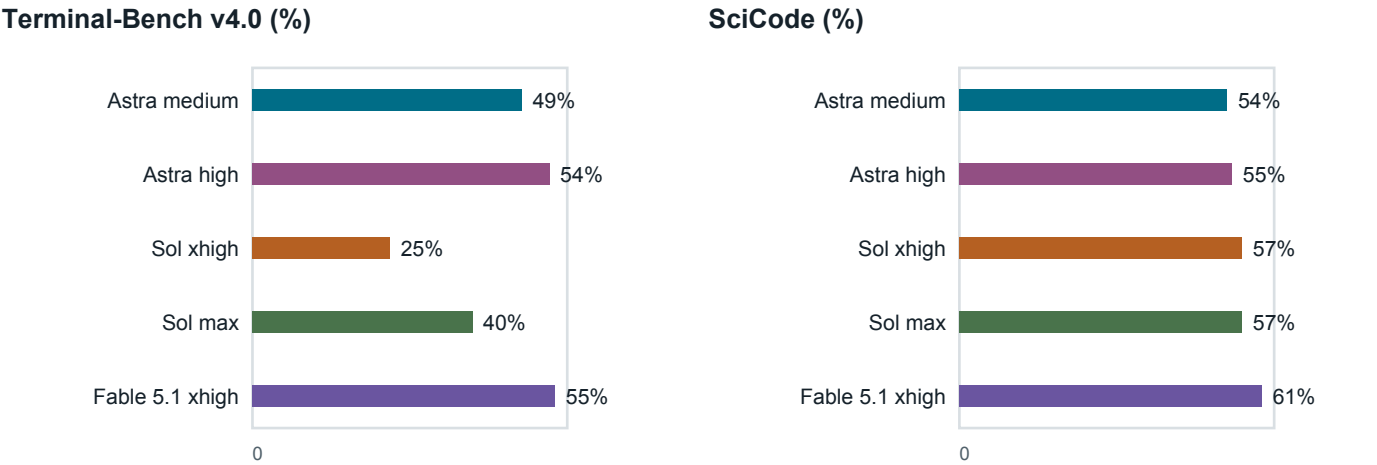


Coding performance, cost and waiting

Expanded edition: Astra medium/high, Sol xhigh/max and Fable 5.1 xhigh, plus the full Astra/Fable effort sweep.

Astra medium remains a sensible interactive starting point. Astra high adds five terminal percentage points and one SciCode point, at about 12% higher mixed-suite cost and 8.8x the first-answer wait in the 9 September snapshot. Escalating for difficult work is a practical judgment, not a validated routing policy. [S1-S3]



Configuration	Output*	Reasoning*	USD*	First answer**
Astra medium	10k	3k	\$1.54	5.21s
Astra high	12k	5k	\$1.72	45.63s
Sol xhigh	20k	10k	\$1.18	50.59s
Sol max	29k	17k	\$1.99	138.22s
Fable 5.1 xhigh	61k	34k	\$5.98	138.44s

Sol: Astra medium scores higher on terminal work than either selected Sol setting, but lower on SciCode. Sol xhigh is the least expensive mixed-suite configuration here. Sol max improves terminal performance over xhigh while increasing output, cost and initial waiting. [S1-S2]

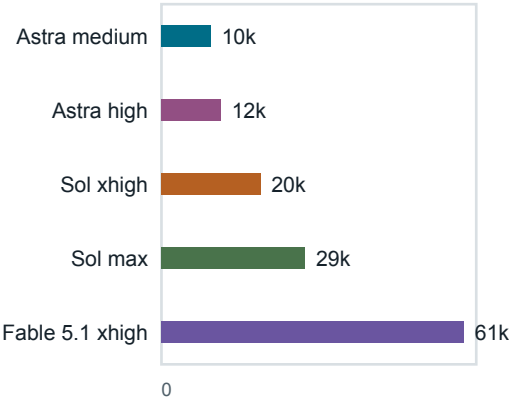
Fable xhigh: the highest displayed score on both coding benchmarks in this five-way selection. Relative to Astra high, its terminal lead is only one rounded percentage point; SciCode leads by six points. No significance claim is supported. Its mixed-suite cost is 3.48x and first-answer wait 3.03x Astra high. [S3]

*Weighted Intelligence Index v4.3 tasks, not coding-only costs. Reasoning is already included in total output. **Separate API performance prompts, not time to finish a tested patch. Fable uses adaptive reasoning with default fallback.

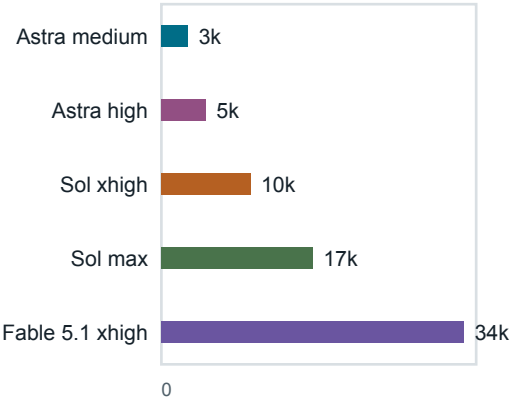
The resource trade-off

The selected configurations use 9 September readings. The original full sweep on pages 3-7 retains its explicitly dated 8 September snapshot.

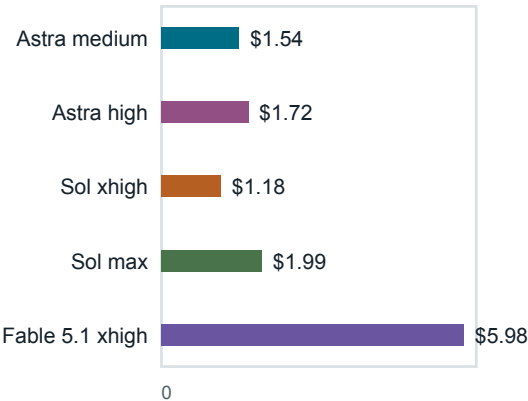
Output / task (thousand tokens)



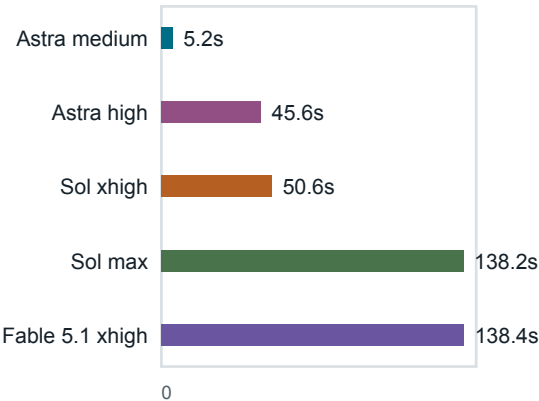
Reasoning subset (thousand tokens)



Mixed-suite cost / task (USD)



First answer (seconds)



Why the dates matter: first-answer latency is a changing API measurement. For example, Fable xhigh is 99.86 seconds in the earlier sweep and 138.44 seconds in the later selection. These are separate snapshots, not a measured regression. The displayed coding scores and cost values agree across the snapshots for the overlapping configurations.

Scope: Sol is included only at the two requested settings. No unmeasured Sol effort curve is inferred. Equal effort names across model families do not specify equal compute budgets. Use actual repository acceptance tests and repeated paired trials to choose a default.

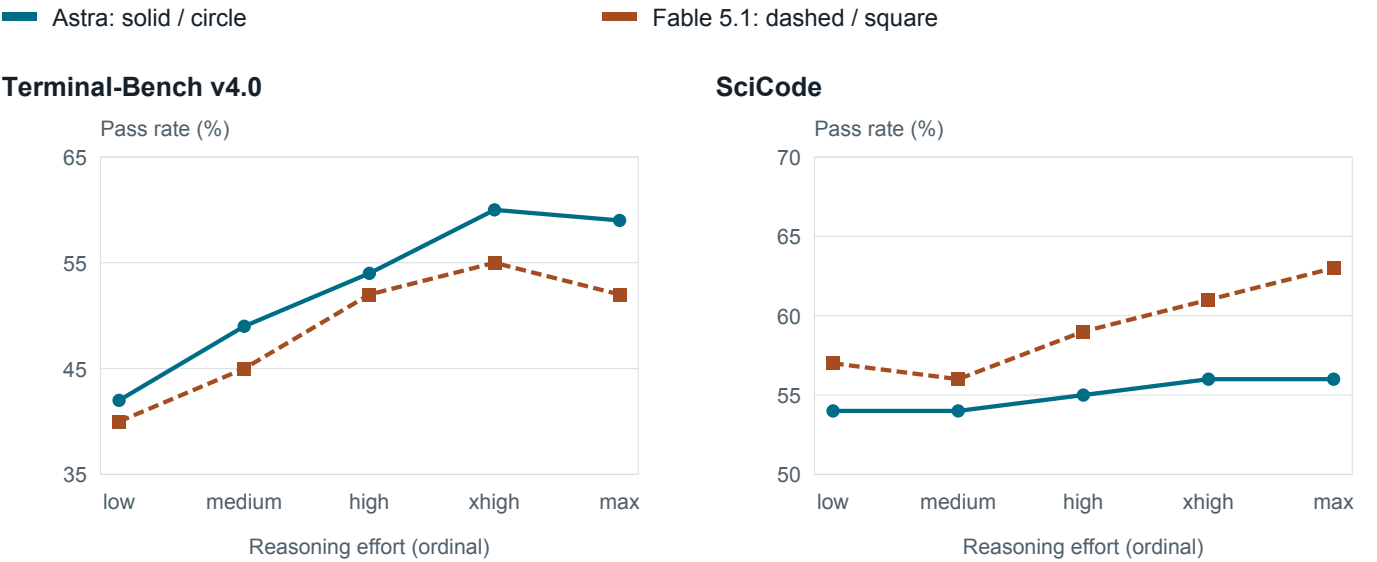
- [S1] [Artificial Analysis: Astra medium / Sol xhigh](#)
- [S2] [Artificial Analysis: Astra medium / Sol max](#)
- [S3] [Artificial Analysis: Astra high / Fable xhigh](#)

The following five pages reproduce and rebuild the original research analysis and vector charts. Their 8 September data and historical interpretations remain distinct from this expanded selection.

More reasoning has a price

GPT-6 Astra and Claude Fable 5.1 across low, medium, high, xhigh and max.

The best setting depends on the coding task. In the 8 September Artificial Analysis sweep, Astra leads on terminal work, while Fable leads on scientific code generation. Both terminal curves peak at xhigh; max adds spending without improving that observed score. This supports selective escalation, not a universal maximum-effort default. [1-6]



Figures 1-2. Whole-percent values from the same five comparison tables. Lines join discrete observations; they are not fitted curves. The two benchmarks measure different kinds of coding. [1-5]

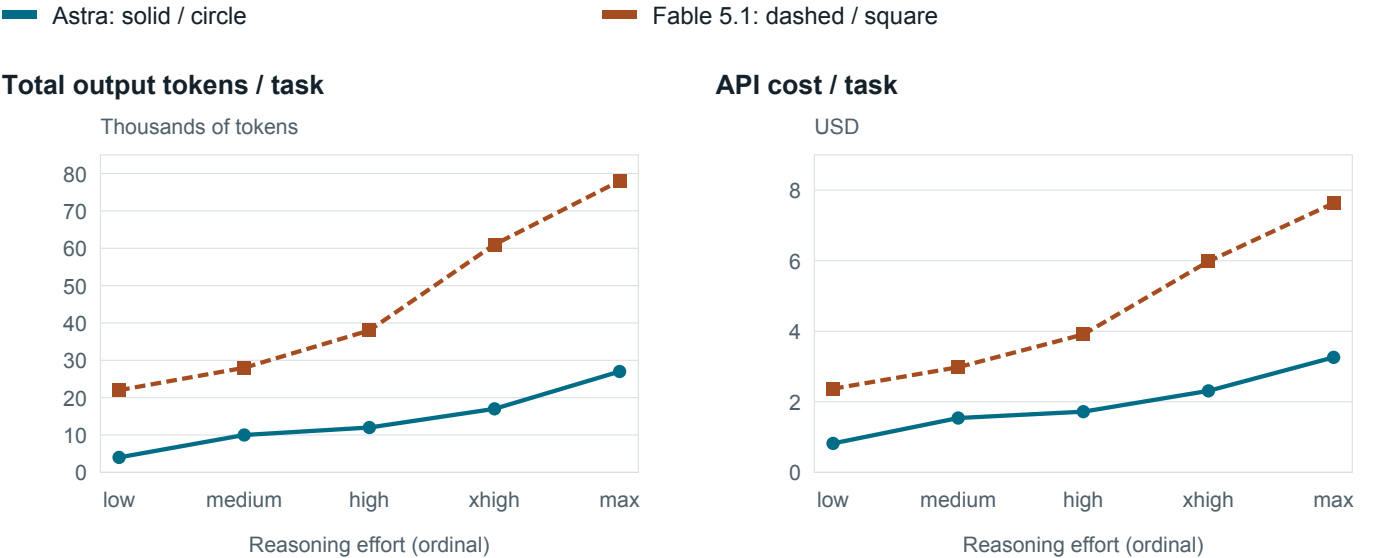
Effort	Astra terminal	Fable terminal	Astra SciCode	Fable SciCode
low	42%	40%	54%	57%
medium	49%	45%	54%	56%
high	54%	52%	55%	59%
xhigh	60%	55%	56%	61%
max	59%	52%	56%	63%

Practical starting point: medium for quick, bounded work; high when correctness deserves more deliberation; xhigh for difficult terminal tasks where waiting is acceptable. Fable max has a more plausible case for scientific coding than for general terminal work. These are workload-dependent judgments from the curves, not tested routing policies.

Evidence boundary: coding scores are benchmark-specific. The token and cost curves on page 4 summarize the broader Intelligence Index; latency on page 5 uses separate API performance prompts. They must not be combined into a claimed measured cost or completion time per successful code change.

Tokens and cost across the ladder

Resource curves are weighted Intelligence Index v4.3 aggregates, not coding-only invoices.



Figures 3-4. Output includes reasoning; do not add reasoning again when calculating the bill. Values in thousands are rounded as published. The source weighting means a suite total divided by a task count will not necessarily reproduce these averages. [1-5]

Model / effort	Output	Reasoning subset	USD / task
Astra / low	4k	0.938k	\$0.82
Astra / medium	10k	3k	\$1.54
Astra / high	12k	5k	\$1.72
Astra / xhigh	17k	9k	\$2.31
Astra / max	27k	17k	\$3.26
Fable 5.1 / low	22k	8k	\$2.37
Fable 5.1 / medium	28k	12k	\$2.98
Fable 5.1 / high	38k	19k	\$3.91
Fable 5.1 / xhigh	61k	34k	\$5.98
Fable 5.1 / max	78k	47k	\$7.63

Astra: low to max raises total output from about 4k to 27k tokens per weighted task (6.75x), and cost from \$0.82 to \$3.26 (3.98x). High to max nearly doubles the cost (1.90x). These resource changes do not imply proportional improvements in coding success.

Fable: low to max raises output from about 22k to 78k (3.55x), and cost from \$2.37 to \$7.63 (3.22x). At high, its cost is 2.27x Astra high in this mixed suite; this is not a 2.27x price-per-token difference. Both list the same base input and output rates. [8-9]

Waiting rises faster than throughput

Time to first answer is an interaction metric, not elapsed time to a tested patch.

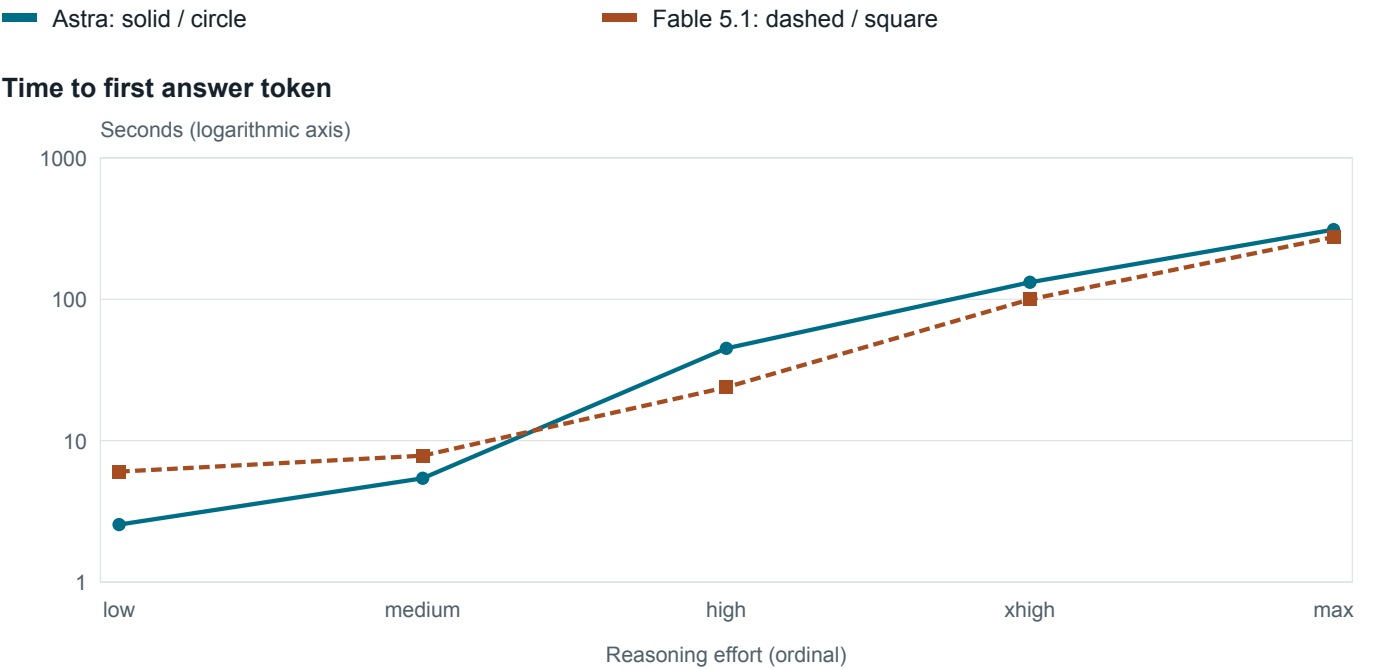


Figure 5. Published API latency snapshot, separate from the coding evaluations. A logarithmic axis makes both short and multi-minute waits visible. The comparison tables show the same value for first token and first answer token; this report uses the latter label. [1-5]

Effort	Astra first answer	Fable first answer	Astra 500-token response	Fable 500-token response
low	2.55s	6.04s	11.72s	15.60s
medium	5.42s	7.83s	14.52s	17.01s
high	44.97s	23.85s	53.75s	32.77s
xhigh	131.86s	99.86s	140.67s	108.48s
max	310.77s	275.52s	319.03s	282.81s

The speed ranking changes with effort. Astra has the shorter first-answer wait at low and medium. Fable has the shorter wait at high, xhigh and max. From high to max the wait grows 6.91x for Astra and 11.55x for Fable. At max, both are above four and a half minutes on this workload.

The 500-token response column is a source-calculated response estimate, not a repository run. It includes the pre-answer wait plus generation of a fixed answer length. It does not include terminal commands, builds, tests, tool round trips, retries or human review.

AA also reports a calculated weighted decode time per task: high/max are 206/457 seconds for Astra and 427/731 seconds for Fable. This excludes first-token time and overhead. Do not add those averages to the first-answer figures above: the workloads and weighting differ, and reasoning time can be counted twice.

Where extra effort earns its keep

A decision guide based on observed trade-offs, with uncertainty retained.

Step	Terminal change	SciCode change	Cost increase*	First-answer increase*
Astra high > xhigh	+6 pp	+1 pp	34%	193%
Astra xhigh > max	-1 pp	0 pp	41%	136%
Fable high > xhigh	+3 pp	+2 pp	53%	319%
Fable xhigh > max	-3 pp	+2 pp	28%	176%

*Cost is the mixed-suite weighted average; first-answer latency is the separate API workload. Percentage-point changes use rounded coding scores. This table aligns effort settings; it does not estimate marginal dollars per coding success. [3-5]

Interactive engineering: high is a reasonable compromise for nontrivial work; medium deserves a trial for routine changes. The jump from medium to high lengthens the initial wait substantially, especially for Astra. Choose using the actual tests and correction rate on your repositories, not the amount of text produced.

Hard terminal work: xhigh is the observed peak for both. Astra xhigh reaches 59.6% in the more precise leaderboard summary, versus 59.1% at max; Fable xhigh reaches 55.1%. A small observed decline is not proof that additional thinking causes harm. No paired trial-level significance test was available for this synthesis. [6]

Scientific coding: Fable keeps gaining through max in the displayed SciCode sweep, while Astra is nearly flat. That makes Fable max worth testing when a scientific implementation is costly to get wrong. A two-point benchmark increase still needs to justify the additional wait and usage on the real workload.

The bill depends on token categories

Standard USD / 1M tokens	Astra	Fable 5.1
Fresh input	\$10	\$10
Cached input read	\$1	\$0.25
Output, including reasoning	\$50	\$50

For disjoint token categories: cost = (10 x fresh input + cache-read rate x cached reads + 50 x total output) / 1,000,000 + applicable cache-write and tool charges. Higher effort changes the quantities, not these base rates. Fable saves \$0.75 per million cache reads relative to Astra; 15,000 extra output tokens would consume that saving. This is arithmetic, not a measured workload. [8-9]

Astra prompts above 272k input tokens have a whole-request surcharge: 2x input/cache rates and 1.5x output. Cache writes, service tiers and tool fees need their own accounting. API dollars are not a conversion of ChatGPT or Claude subscription quotas. [8]

Methods, limits and sources

Research snapshot: 8 September 2026. No paid evaluation was run for this report.

The 8 September dataset uses five Artificial Analysis comparison tables, preserving their displayed precision. Terminal-Bench uses mini-SWE-agent v2.4.6, 66 tasks and three repeats; SciCode uses 288 scientific Python subproblems and three repeats. The Intelligence Index covers ten evaluations and is not a coding-success percentage. Fable entries include default safety fallback to other Claude models. [1-7,11]

Version matters: v4.3 replaced Terminal-Bench v2.1 with v4.0. Older 89.9-91.4% Fable scores belong to v2.1 and must not be plotted against these v4.0 scores. The earlier workspace report used an older index. Suite-token totals also differ slightly between live model and comparison pages; the curves here use the per-task comparison rows throughout. [7]

Adjacent evidence: Firecrawl reports a high-effort Fable agentic-build mean of 79,068 output tokens, 18 minutes and \$7.00 across only two builds. Its small prompt experiment also shows substantial run-to-run variance. This supports workload sensitivity but cannot establish an Astra-versus-Fable latency curve or a general optimum. [12]

To validate a default locally, run the same representative tasks and repository starting states at each effort, with fixed tools and acceptance tests. Randomize run order and repeat tasks. Record fresh input, cache writes/reads, reasoning, total output, cost, first answer, elapsed completion time and verified pass/fail. Compare paired outcomes with task-level uncertainty; include failures and retries in total spending. This is a proposed protocol, not completed evidence.

Source register

The following links were consulted for the original 8 September research. AA = Artificial Analysis. References 1-5 supply each effort row; 6-7 establish benchmark definitions and methodology.

- [1] [AA: low-effort comparison](#)
- [2] [AA: medium-effort comparison](#)
- [3] [AA: high-effort comparison](#)
- [4] [AA: xhigh-effort comparison](#)
- [5] [AA: max-effort comparison](#)
- [6] [AA: Terminal-Bench v4.0](#)
- [7] [AA: intelligence methodology](#)
- [8] [OpenAI: Astra model and pricing](#)
- [9] [Anthropic: Fable and pricing](#)
- [10] [Anthropic: Fable 5.1 launch](#)
- [11] [AA: Fable launch evaluation and fallback](#)
- [12] [Firecrawl: small-sample API measurements](#)

Interpretation limits: five ordinal settings do not identify a continuous scaling law or a causal mechanism. Equal rounded scores can hide small differences. Sampling, task difficulty, fallback, tokenization, cache policy and harness behavior affect results. A plateau does not prove that all remaining errors are impossible to solve with more reasoning.